Building a Custom Model Quantizer in PyTorch

Author: Abigail Tarekegn
Model: HuggingFaceTB/SmolLM2-135M
Environment: Google Colab, CPU runtime

This project builds a custom model quantizer in raw PyTorch without using pre-built quantization libraries. The selected model is **SmolLM2-135M**, and the goal is to reduce its inference memory requirements while preserving prediction quality. The target is to keep the inference program's peak RAM usage below **1 GB**, including the model, supporting libraries, and temporary inference data.

The implementation calculates scales and zero points, rounds and clamps integer codes, and packs two INT4 codes into each byte. During inference, each layer reconstructs its weights in FP32 before computation. This saves weight storage but introduces unpacking and reconstruction costs.

This Model quantization project involves three controlled comparisons:

1. **Bit width:** INT8 versus INT4, both per-channel symmetric.
2. **Granularity:** per-tensor versus per-channel, both INT4 symmetric.
3. **Mapping:** symmetric versus asymmetric, both INT4 per-channel.

How the models were evaluated

Every configuration received the same held-out text and generation prompt. Evaluation covered four practical questions:

- **How much disk space does it occupy?** The saved weight checkpoint was measured in MB, including quantization metadata stored with the tensors.
- **How much RAM does it need?** Each model ran in a fresh Python process. Physical RAM was sampled every 5 milliseconds during text evaluation and generation. The highest observed reading was compared with the 1,000 MB budget.
- **How well does it predict text?** Each model predicted the next tokens in the same WikiText-2 test sample: 16 blocks of 128 tokens, producing 2,032 evaluated predictions. Perplexity summarizes how well the model assigns probability to the actual next tokens. *Lower is better; perplexity is not an accuracy percentage.

- **How long does generation take?** Each model generated 50 new tokens from the same prompt using greedy decoding. After one warmup, three runs were timed and the median was reported. This timing excluded model loading and memory sampling.

Measured results

Configuration	Checkpoint MB	Peak RAM MB	Weight MSE	Perplexity ↓	Time for 50 tokens, s ↓
FP32 baseline	538.10	1,105.4	0	36.203	3.700
INT8 channel symmetric	220.23	788.7	0.000002698	36.585	8.704
INT4 channel symmetric	167.15	730.7	0.0008829623	81.945	23.265
INT4 tensor symmetric	166.52	729.6	0.012556351	4,976,543.760	20.564
INT4 channel asymmetric	167.33	730.6	0.000653875	65.510	20.419

All quantized configurations met the RAM target; the FP32 baseline did not.

Interpretation of the three experiments

Bit width: INT8 reduced checkpoint size by approximately **59.1%** and peak RAM by **28.7%** relative to FP32. Its perplexity increased by only **1.1%**. Moving to symmetric per-channel INT4 saved another 58 MB of RAM, but perplexity increased to 81.945 and generation became substantially slower.

Granularity: INT4 per-tensor saved less than 1 MB compared with per-channel INT4, but prediction quality deteriorated severely. One shared scale per matrix was unsuitable for this model under the tested scheme. Per-channel scaling was clearly preferable.

Mapping: Asymmetric per-channel INT4 improved perplexity from 81.945 to 65.510 with almost unchanged RAM and checkpoint size. However, its perplexity remained approximately **81.0% above the FP32 baseline**. It was the better INT4 configuration by measured perplexity, but INT8 preserved quality much better.

Final decision and limitations

INT8 per-channel symmetric was selected. It met the 1 GB RAM constraint while preserving prediction quality most closely. The checkpoint shrank from 538.10 MB to 220.23 MB, and peak inference RAM fell from 1,105.4 MB to 788.7 MB. The cost was slower generation: 8.704 seconds versus 3.700 seconds for 50 tokens, approximately **2.35 times as long**.

The results show that storage savings, RAM savings, quality, and speed are separate outcomes. This custom implementation reconstructs floating-point weights during inference, so lower-bit storage did not deliver faster generation. The small evaluation sample, sampled memory measurements, single prompt, and cloud hardware limit generalization. The findings do not establish that optimized quantization implementations would show the same speed trade-off

Implementation hurdles, failures and corrections

One problem faced during implementation is choosing the right model. I first considered Pythia-70M, but its input embeddings and output projection account for much of its weight storage, limiting savings when only internal layers are quantized. The reason I wanted to focus my quantization on internal layers is that leaving the input embeddings and output projection unchanged avoids introducing quantization error at those stages, allowing me to study the effects of bit width, scaling granularity, and symmetric versus asymmetric quantization within a consistent scope. The second model I considered was SmolLM2-135M because a larger proportion of its parameters lie in the internal linear layers targeted by the quantizer. The problem I faced with this one is that my laptop has 8 GB of installed RAM but only about 0.8 GB currently available, so TinyStories-8M was considered as a smaller alternative. Although TinyStories-8M is smaller, its embeddings account for a larger share of its parameters, which limits the overall memory savings from this approach. In SmolLM2-135M, roughly 79% of the parameters are in internal linear layers, giving the quantizer more weights to compress and making the trade-offs between bit width, granularity, and quantization mapping easier to investigate. TinyStories would be more convenient for my laptop's limited RAM, but running SmolLM2 on a cloud CPU lets me study the model better suited to my chosen experimental scope.

The INT4 per-tensor configuration was a substantive experimental failure: it compressed successfully and ran, but its extremely high perplexity made it unsuitable.

Submission links:

GitHub: https://github.com/abigailtassefa/smollm2-pytorch-quantizer

Hugging Face: https://huggingface.co/Abbyxx/smollm2-135m-custom-int8

.